Parameter Estimation

Charlie Shields

1 Reading File

```
f = open ("data1.txt","r")
energy =[ ]; Npion =[ ]; Npionerr =[ ]
for line in f :
    estr , nstr , errstr = line . split ()
    energy . append ( float ( estr ))
    Npion . append ( float ( nstr ))
    Npionerr . append ( float ( errstr ))
f . close ()
energy = array ( energy )
Npion = array ( Npion )
Npionerr = array ( Npionerr )
```

This code opens the data file (data1.txt) and reads each line adding the 3 values onto seperate arrays for Energy, Number of Pions and the error in Number of Pions.

2 Plotting Data

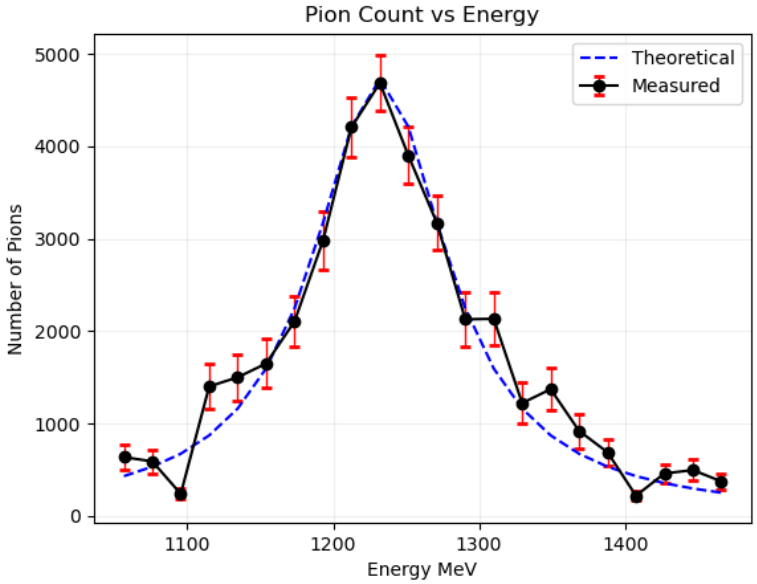


Figure 1: Measured number of pions vs energy with theoretical curve.

Q2.2 The plotted data displays a symmetrical, bell-shaped curve, with a peak of 4571.9 pions at 1214 MeV. The graph deviates more from the theoretical values at the higher and lower energies, where the curve gradually flattens out.

```
plt.errorbar(
    energy, Npion, yerr=Npionerr,
    color = "black",
    fmt = "o-",
    ecolor= "red",
    elinewidth=1,
    capsize=3,
    capthick=2,
label = "Measured")
plt.xlabel("Energy MeV")
plt.ylabel("Number of Pions")
plt.grid(True,alpha=0.2)
plt.title("Pion Count vs Energy")
```

Above shows the code used to plot the data using Matplotlib with the uncertainty in the number of pions being used to make error bars. Theoretical values were also calculated and plotted along the data for comparison which was done by using an estimated value of w calculated using the middle value of energy and number of pions from the data. The code for this is below:

```
def theory(E, w,):

    return (( 1.467*10**7)/  ((w**2/4 )+(E - 1232)**2)  )

E_mid = energy[len(energy) // 2]
n_mid = Npion[len(energy) // 2]

w = sqrt(  4*( (1.467*10**7/n_mid) - (E_mid - 1232)**2   )  )
th = [ ]
th = theory(energy, w)

plt.plot(energy, th, "b--", label="Theoretical")
plt.legend()
plt.savefig("PionCountvEnergy_Plot.png")
plt.show()
```

Q2.3 This code gave a value for w to be 111.3798 and by plotting greater and smaller values of w manually shows a higher w value decreases the size of the theoretical peak with a lower value increasing the size.

Q2.5 Using curve fit allows us to find the value for w that makes the theoretical model most closely fit with the experimental data. It does this by changing the value of w around a given estimate until the data fits the best. For our estimate the value for w found before was used along with both sets of data the error in the experimental data. Curve fit outputs two arrays with a value of w with its uncertainty. The value of w obtained from curve fit was 113.6313 +- 2.2386.

```
popt, pcov = curve_fit(theory, energy, Npion, sigma = Npionerr, p0 = w, absolute_sigma = True)

w_optimal = popt[0]
w_err = sqrt(pcov[0,0])
print(f"Optimal w from Curve fit = {w_optimal:.4f} +- {w_err:.4f}")
```

3 Random Sampling

Q2.6 In order to create a random sample a for loop is used which repeatedly creates a random distribution within the uncertainty of the number of pions and used curve fit again using these distributions to create an array of w values of which the mean is an estimate with the standard deviation a measure of the uncertainty.

```
for i in range(1000):
    simulated_data = random.normal(Npion, Npionerr)

    popt, _ = curve_fit(theory, energy, simulated_data, p0=[w])
    w_values.append(popt[0])

w_mean = mean(w_values)
w_std = std(w_values)
```

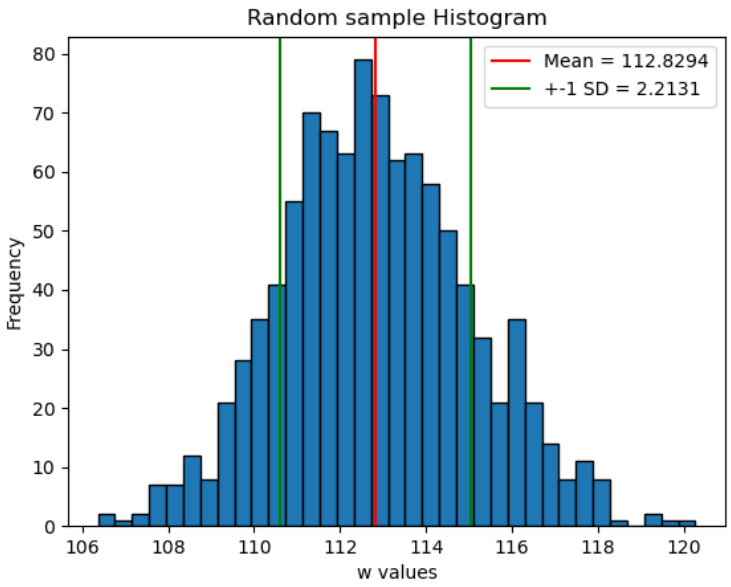


Figure 2: Histogram of random sample w values with mean and standard deviation

Q2.7 The histogram shows a rough bell curve distribution centred at the mean 112. The relatively small uncertainty of 2.2 with the narrow spread of the histogram suggests the value of w is reliable. This is supported by similar values and uncertainties to the curve fits parameter estimation.

```
plt.hist(w_values, bins = 35, edgecolor= "black") ## changes how many bars on histogram
plt.xlabel("w values")
plt.ylabel("Frequency")
plt.title("Random sample Histogram" )
plt.axvline(w_mean, color = "red" , label=f"Mean = {w_mean:.4f}") ## adds a line for the mean
plt.axvline(w_mean - w_std, color="green", label=f"+-1 SD = {w_std:.4f}")
plt.axvline(w_mean + w_std, color="green")
plt.legend()
plt.savefig("Histogram_Plot.png")
plt.show()
```

Above shows the code for the histogram, the bins value used of 35 determines the width of each bar in the histogram, 35 was sued as it demonstrated the bell curve of the data while having visible bars.